

使用 Google Gemini 檔案搜尋工具進行檢索增強生成

來源：<https://codelabs.developers.google.com/gemini-file-search-for-rag?hl=zh-tw>

步驟數：8

本程式碼研究室說明如何使用 Gemini 檔案搜尋，在 Agentic 應用程式中啟用 RAG。您可以使用 Gemini 檔案搜尋功能擷取文件並建立索引，不必擔心分塊、嵌入或向量資料庫的詳細資料。

目錄

1. [簡介](#)
2. [什麼是 RAG ? 為什麼需要 RAG ?](#)
3. [必要條件](#)
4. [基本虛擬服務專員](#)
5. [Gemini 檔案搜尋功能說明](#)
6. [如何使用 Gemini 檔案搜尋功能 ?](#)
7. [將代理轉換為使用 ADK](#)
8. [結論](#)

1. 簡介

本程式碼研究室說明如何使用 [Gemini 檔案搜尋](#)，在代理程式應用程式中啟用檢索增強生成 (RAG)。您可以使用 Gemini 檔案搜尋功能擷取文件並建立索引，不必擔心分塊、嵌入或向量資料庫的詳細資料。

課程內容

- RAG 的基本概念，以及我們為何需要 RAG。
- 瞭解 Gemini 檔案搜尋功能和優點。
- 如何建立檔案搜尋儲存庫。
- 如何將自訂檔案上傳至檔案搜尋商店。
- 如何使用 Gemini 檔案搜尋工具進行檢索增強生成。
- 使用 Google Agent Development Kit (ADK) 的優點。
- 如何使用 ADK 建構的代理程式解決方案中的 Gemini 檔案搜尋工具。
- 如何搭配使用 Gemini 檔案搜尋工具和 Google 「原生」工具 (例如 Google 搜尋)。

學習內容

- 建立 Google Cloud 專案並設定開發環境。
 - 使用 Google Gen AI SDK (但不使用 ADK) 建立簡單的 Gemini 代理，該代理可使用 Google 搜尋，但不具備 RAG 功能。
 - 證明無法提供準確的高品質資訊。
 - 建立 Jupyter 筆記本 (可在本機或 Google Colab 等平台執行)，用於建立及管理 Gemini 檔案搜尋儲存庫。
 - 使用筆記本將自訂內容上傳至檔案搜尋儲存庫。
 - 建立附加檔案搜尋儲存空間的代理，並證明代理可以生成更理想的回覆。
 - 將初始「基本」代理轉換為 ADK 代理，並完成 Google 搜尋工具。
 - 使用 ADK 網頁 UI 測試代理程式。
 - 使用「代理即工具」模式，將檔案搜尋儲存庫併入 ADK 代理，以便同時使用檔案搜尋工具和 Google 搜尋工具。
-

2. 什麼是 RAG？為什麼需要 RAG？

所以... 檢索增強生成。

您來到這裡，應該知道這是什麼，但我們還是快速複習一下。大型語言模型 (例如 Gemini) 非常聰明，但有幾個問題：

1. 資訊永遠過時：模型只會知道訓練期間學到的內容。
2. 模型並非無所不知：模型雖然龐大，但並非無所不知。
3. 不會知道您的專屬資訊：雖然知識廣博，但不會讀取您的內部文件、網誌或 Jira 支援單。

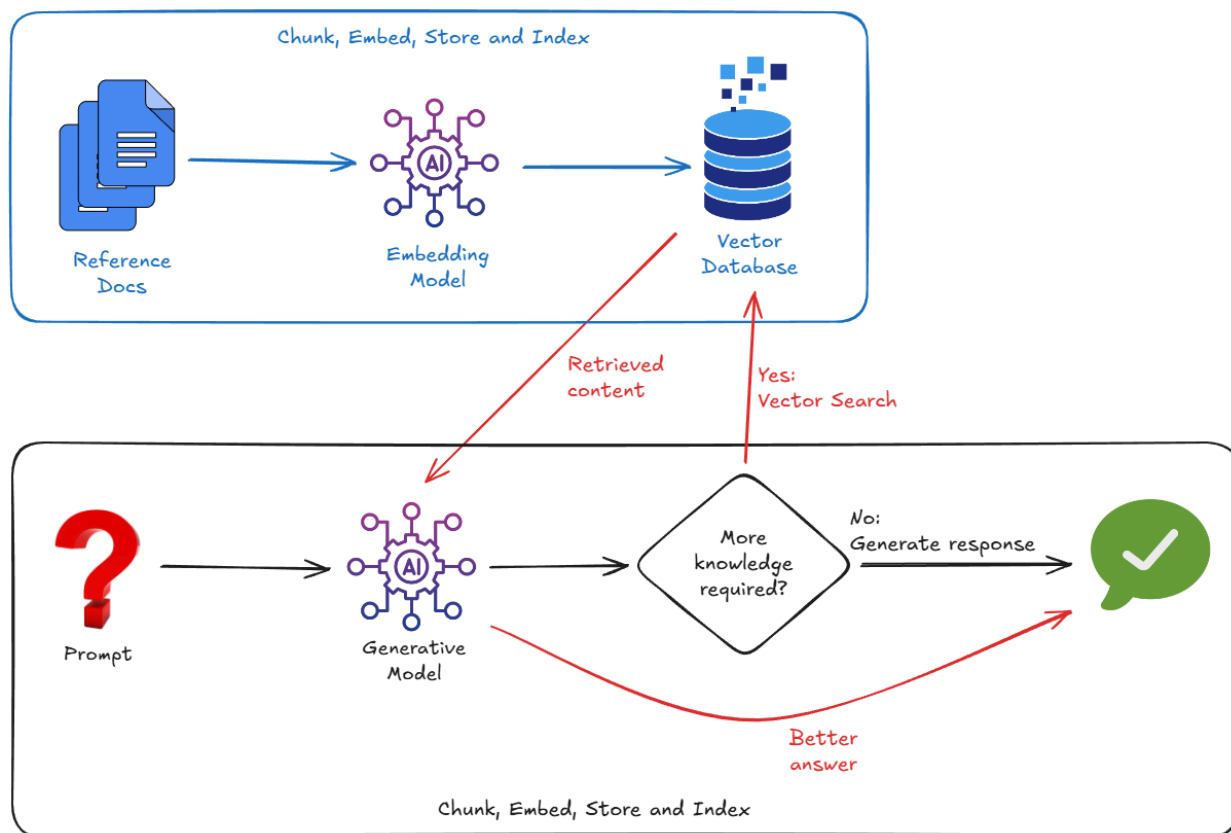
因此，當你詢問模型不知道答案的問題時，模型通常會提供錯誤或甚至編造的答案。模型通常會自信地給出錯誤答案。這就是所謂的「幻覺」。

其中一個解決方法是直接將專有資訊 *傾印* 到對話內容中。如果資訊量不大，這種做法還算可行，但如果資訊量很大，就會迅速產生問題。具體來說，這會導致下列問題：

1. 延遲：模型回覆速度越來越慢。
2. 訊號腐化，又稱「中間遺失」：模型無法再從垃圾資料中排序相關資料。模型會忽略大部分的脈絡資訊。
3. 費用：因為權杖會產生費用。
4. 脈絡窗口耗盡：此時 Gemini 不會執行您的要求。

如要更有效地解決這個問題，請使用 **RAG**。這項程序只是從您的資料來源 (使用語意比對) 查詢相關資訊，並將這些資料的相關區塊連同您的問題提供給模型。根據實際情況建立模型基準。

這項技術的運作方式是匯入外部資料、將資料切分成塊、將資料轉換為向量嵌入，然後將這些嵌入項目儲存並編入合適的向量資料庫。



因此，如要實作 RAG，通常必須注意以下事項：

1. 啟動向量資料庫 (Pinecone、Weaviate、Postgres with pgvector...)。

2. 編寫分塊指令碼，將文件 (例如 PDF、Markdown 等) 分割成多個部分。
3. 使用嵌入模型為這些區塊生成嵌入 (向量)。
4. 將向量儲存在向量資料庫中。

但朋友不會讓朋友過度設計。如果我告訴你有一種更簡單的方法呢？

3. 必要條件

建立 Google Cloud 專案

您需要 Google Cloud 專案才能執行本程式碼研究室。您可以使用現有專案，或[建立新專案](#)。

請確認專案已啟用[帳單](#)。如要瞭解如何查看專案的帳單狀態，請參閱[這份指南](#)。

請注意，完成本程式碼研究室不會產生任何費用。最多只會收取幾分錢。

請準備好專案。我會等。

複製示範存放區

我已建立存放區，其中包含本程式碼研究室的引導式內容。之後一定能派上用場！

從終端機或整合到 Google [Cloud Shell 編輯器](#)的終端機執行下列指令。Cloud Shell 及其編輯器非常方便，因為所有必要指令都已預先安裝，一切都可「開箱即用」。

```
git clone https://github.com/derailed-dash/gemini-file-search-demo
cd gemini-file-search-demo
```

這個樹狀結構會顯示存放區中的重要資料夾和檔案：

```
gemini-file-search-demo/
├── app/
│   ├── basic_agent_adk/           # Agent with Google Search, using ADK framework
│   │   └── agent.py
│   ├── rag_agent_adk/           # Agent with Google Search and File Search, using
ADK framework
│   │   ├── agent.py
│   │   └── tools_custom.py
│   └── sdk_agent.py              # Agent using GenAI SDK (no ADK) with Google Search
tool
│   └── sdk_rag_agent.py          # Agent using GenAI SDK (no ADK) with Gemini File
Search tool
├── data/
│   └── story.md                  # Sample story with "bespoke content" to use with
Gemini File Search Store
├── notebooks/
│   └── file_search_store.ipynb  # Jupyter notebook for creating and managing Gemini
File Search Store
├── .env.template                 # Template for environment variables - make a copy
as .env
├── Makefile                     # Makefile for `make` commands
├── pyproject.toml               # Project configuration and dependencies
└── README.md                   # This file
```

在 Cloud Shell 編輯器或偏好的編輯器中開啟這個資料夾。(你用過 Antigravity 嗎？如果還沒試過，現在正是[好時機](#)。)

請注意，存放區的 `data/story.md` 檔案中包含範例故事「The Wormhole Incursion」。這篇文章是我和 Gemini 共同撰寫的！故事主角是達茲波指揮官和他的智慧星艦中隊。(我從《Elite Dangerous》這款遊戲獲得一些靈感。)這個故事是我們的「專屬知識庫」，包含 Gemini 不知道的特定事實，而且 Gemini 無法使用 Google 搜尋尋找這些事實。

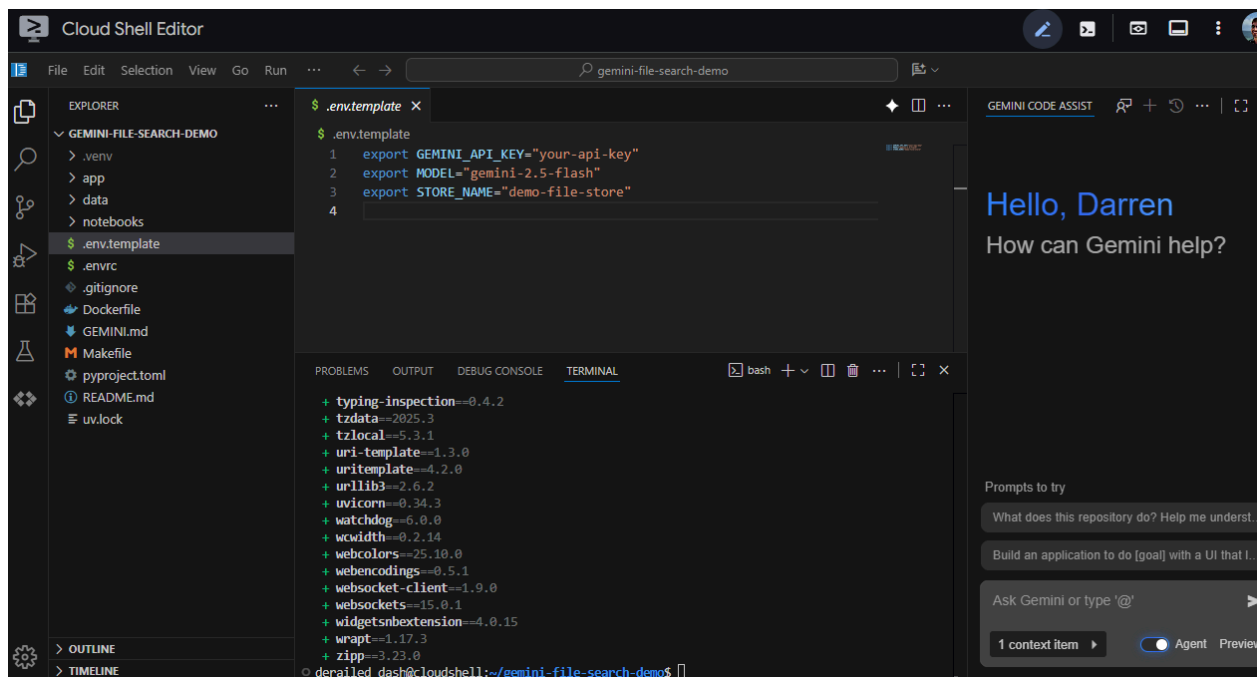
設定開發環境

為方便起見，我提供了一個 `Makefile`，可簡化許多需要執行的指令。不必記住特定指令，只要執行類似 `make <target>` 的指令即可。不過，`make` 僅適用於 Linux / MacOS / WSL 環境。如果您使用 Windows (不含 WSL)，則需要執行 `make` 目標包含的完整指令。

```
# Install dependencies with make
make install

# If you don't have make...
uv sync --extra jupyter
```

在 Cloud Shell 編輯器中執行 `make install` 時，畫面會顯示以下內容：



建立 Gemini API 金鑰

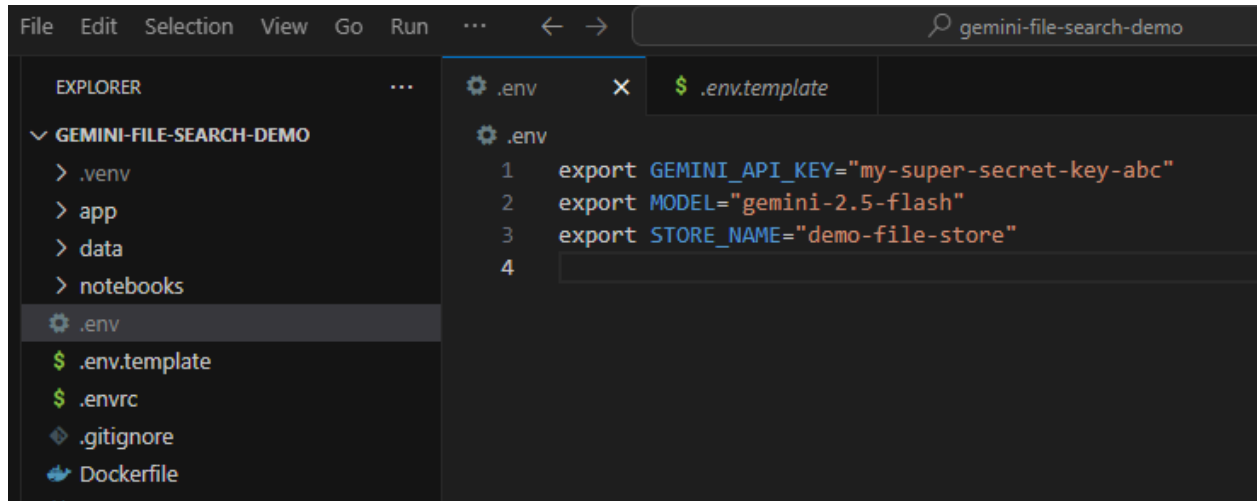
如要使用 Gemini Developer API (使用 Gemini 檔案搜尋工具時需要這個 API)，您必須取得 Gemini API 金鑰。取得 API 金鑰最簡單的方法是使用 [Google AI Studio](#)，這個工具提供便利的介面，可為 Google Cloud 雲端專案取得 API 金鑰。如需要具體步驟，請參閱[這份指南](#)。

建立 API 金鑰後，請複製並妥善保管。

現在需要將這個 API 金鑰設為環境變數。我們可以使用 `.env` 檔案完成這項操作。將隨附的 `.env.example` 複製到名為 `.env` 的新檔案。檔案應會如下所示：

```
export GEMINI_API_KEY="your-api-key"
export MODEL="gemini-2.5-flash"
export STORE_NAME="demo-file-store"
```

請將 `your-api-key` 替換成實際的 API 金鑰。現在畫面應如下所示：



現在請確認環境變數已載入。方法如下：

```
source .env
```


步驟 4 · 約 0 分鐘

4. 基本虛擬服務專員

首先，請建立基準。我們將使用原始 `google-genai` SDK 執行簡單的代理程式。

程式碼

請看一下 `app/sdk_agent.py`。這是最簡單的實作方式，可：

- 將 `genai.Client` 執行個體化。
- 啟用「`google_search`」工具。
- 就是這麼簡單！沒有 RAG。

請詳閱程式碼，確保瞭解其作用。

執行

```
# With make
make sdk-agent

# Without make
uv run python app/sdk_agent.py
```

我們來問一個一般問題：

```
> What is the stock price of Google?
```

這項功能應會使用 Google 搜尋尋找目前價格，並正確回答：

```
(gemini-file-search-demo) derailed_dash@cloudshell:~/gemini-file-search-demo$ make sdk-agent
uv run python app/sdk_agent.py
2026-01-10 18:25:33,660 - INFO - Starting chat with model: gemini-2.5-flash
--- Simple Agent (Model: gemini-2.5-flash) ---
Type 'exit' or 'quit' to stop.
-----
User: What is the stock price of Google?
2026-01-10 18:25:47,069 - INFO - AFC is enabled with max remote calls: 10.
2026-01-10 18:25:49,679 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta
/models/gemini-2.5-flash:generateContent "HTTP/1.1 200 OK"
Agent: As of January 9, 2026, the stock price for Alphabet (Google) is as follows:

*   **Alphabet Class C (GOOG):** The stock trades at approximately $326.04. On January 9, 2026, it t
raded between a low of $326.25 and a high of $331.48. The current price is 329.14 USD, an increase o
f 0.96% in the past 24 hours.
*   **Alphabet Class A (GOOGL):** The current price stands at $322.34. On the most recent trading da
y, the open price was $326.55, with a high of $330.83 and a low of $325.80.
User: █
```

現在，我們來問一個 Gemini 無法回答的問題。這項功能需要代理程式讀取我們的故事。

```
> Who pilots the 'Too Many Pies' ship?
```

這時模型應該會失敗，甚至產生錯誤資訊。請參閱以下範例：

```
User: Who pilots the 'Too Many Pies' ship?
2026-01-10 18:38:05,465 - INFO - AFC is enabled with max remote calls: 10.
2026-01-10 18:38:09,346 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta
/models/gemini-2.5-flash:generateContent "HTTP/1.1 200 OK"
Agent: I could not find any information about a ship named 'Too Many Pies', either real or fictional
, or who pilots it. The search results primarily refer to:

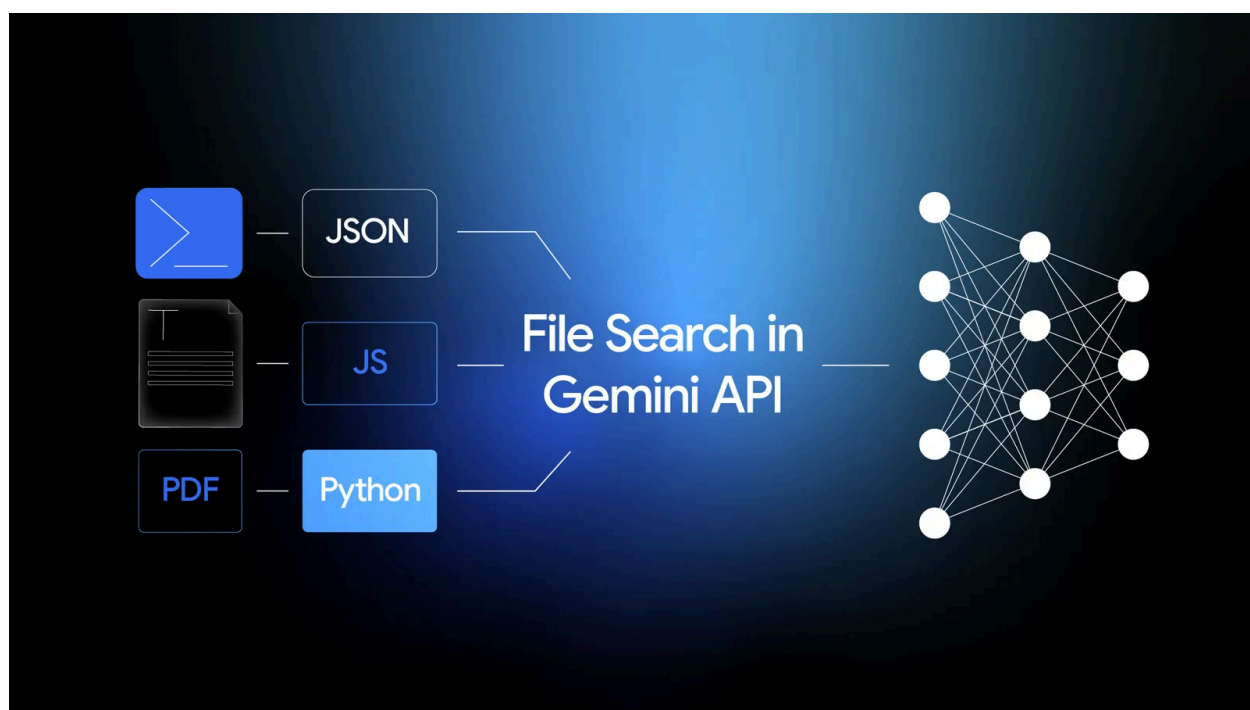
* The process of shipping actual pies or baked goods.
* A popular football chant, "Who Ate All the Pies?" directed at overweight individuals.
* A humorous take on becoming a "pielot" (a pilot who eats many pies).
* A restaurant named "Mama's TOO!" that sells various types of pizza pies.
* A character named Spike from "My Little Pony" who sends notes into pies with his fire breath, bu
t this does not involve a ship.

If 'Too Many Pies' refers to something specific you have encountered, please provide more context, s
uch as the source (e.g., a book, game, movie, or specific story), and I would be happy to try and fi
nd more information.
User: 
```

結果模型果然無法回答問題。完全不知道我們在說什麼！

現在輸入 `quit` 即可退出代理程式。

5. Gemini 檔案搜尋功能說明



Gemini 檔案搜尋功能基本上結合了兩項功能：

1. **全代管 RAG 系統**：您提供大量檔案，Gemini 檔案搜尋會為您處理分塊、嵌入、儲存和向量索引作業。
2. 代理程式中的「工具」：您只要在代理定義中新增 **Gemini 檔案搜尋工具**，並將工具指向檔案搜尋儲存庫即可。

但最重要的是：這項功能內建於 **Gemini API 本身**。也就是說，您不需要啟用任何其他 API 或部署任何獨立產品，就能使用這項功能。因此，這項功能 *out-of-the-box*。

Gemini 檔案搜尋功能

其功能包括：

- 系統會為您 (開發人員) 抽象化分塊、嵌入、儲存和建立索引的詳細資料。也就是說，您不需要瞭解 (或在意) 嵌入模型 (順帶一提，這是 Gemini Embeddings)，也不必知道產生的向量儲存在何處。您不必做出任何向量資料庫決策。
- 這項服務支援大量的文件類型，包括但不限於：PDF、DOCX、Excel、SQL、JSON、Jupyter 筆記本、HTML、Markdown、CSV，甚至是 ZIP 檔案。如需完整清單，請參閱這裡。舉例來說，如果想讓代理程式以含有文字、圖片和表格的 PDF 檔案為基礎，就不必預先處理這些 PDF 檔案。只要上傳原始 PDF，剩下的就交給 Gemini 代勞。
- 我們可以為任何上傳的檔案新增自訂中繼資料。這項功能在執行階段篩選工具要使用的檔案時，非常實用。

資料儲存位置

上傳部分檔案。Gemini 檔案搜尋工具已取得這些檔案、建立區塊和嵌入內容，並將這些內容儲存在某處。但要怎麼做？

答案：**檔案搜尋商店**。這是全代管的嵌入內容容器，您不需要瞭解 (或在意) 幕後作業的執行方式。您只需要以程式輔助方式建立一個，然後將檔案上傳至其中即可。

價格實惠！

儲存及查詢嵌入內容不會產生費用。因此您可以無限期儲存嵌入內容，而且不必支付儲存費用！

事實上，您唯一需要支付的費用，是在上傳/建立索引時建立嵌入內容。**撰寫本文時**，每 100 萬個權杖的費用為 \$0.15 美元。這相當便宜。

步驟 6 · 約 10 分鐘

6. 如何使用 Gemini 檔案搜尋功能？

這項程序分為兩個階段：

1. 在檔案搜尋儲存庫中建立及儲存嵌入項目。
2. 從代理查詢檔案搜尋儲存庫。

第 1 階段 - 使用 Jupyter Notebook 建立及管理 Gemini 檔案搜尋儲存庫

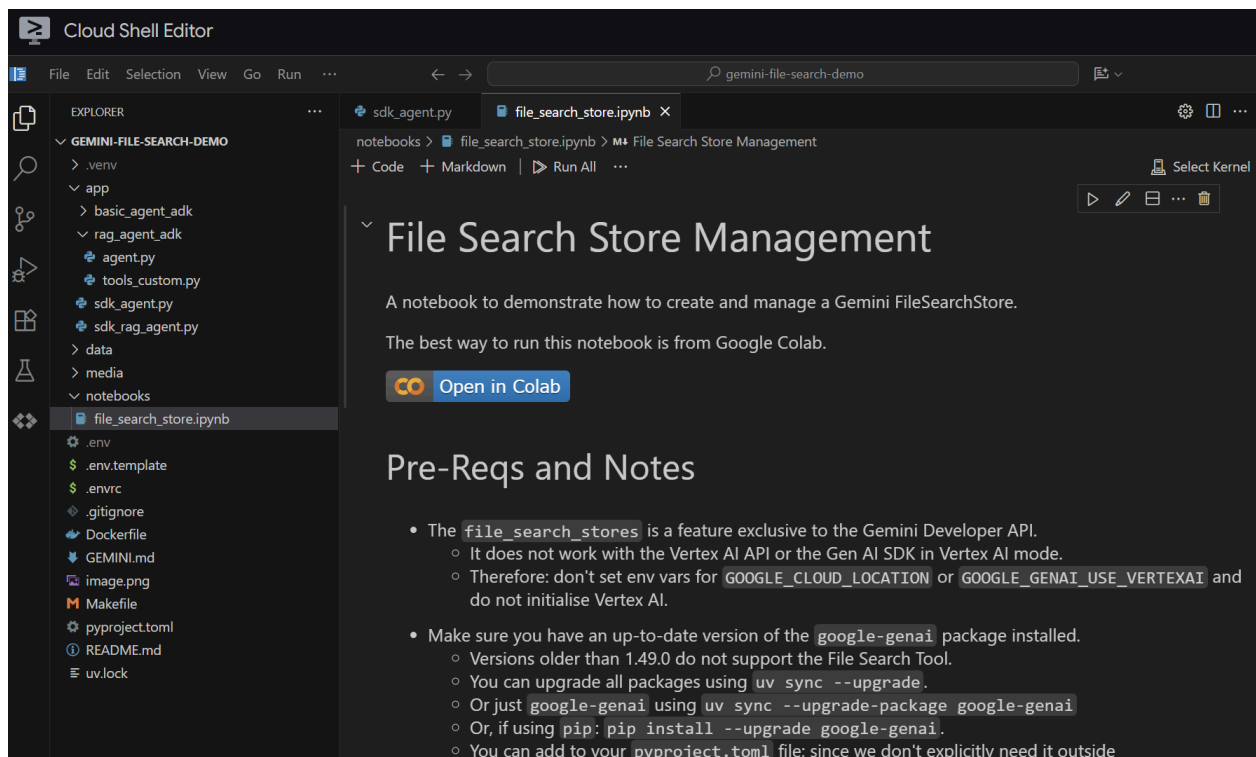
這個階段是初始階段，之後每當你想更新商店時，也需要進行這個階段。舉例來說，當您有新文件要新增，或是來源文件有變更時。

您不需要將這個階段封裝到部署的代理程式應用程式。當然可以，舉例來說，如果您想為代理程式應用程式的管理員使用者建立某種 UI，但通常只要有可隨選執行的程式碼片段就已足夠。而隨選執行這段程式碼的絕佳方式之一，Jupyter 筆記本！

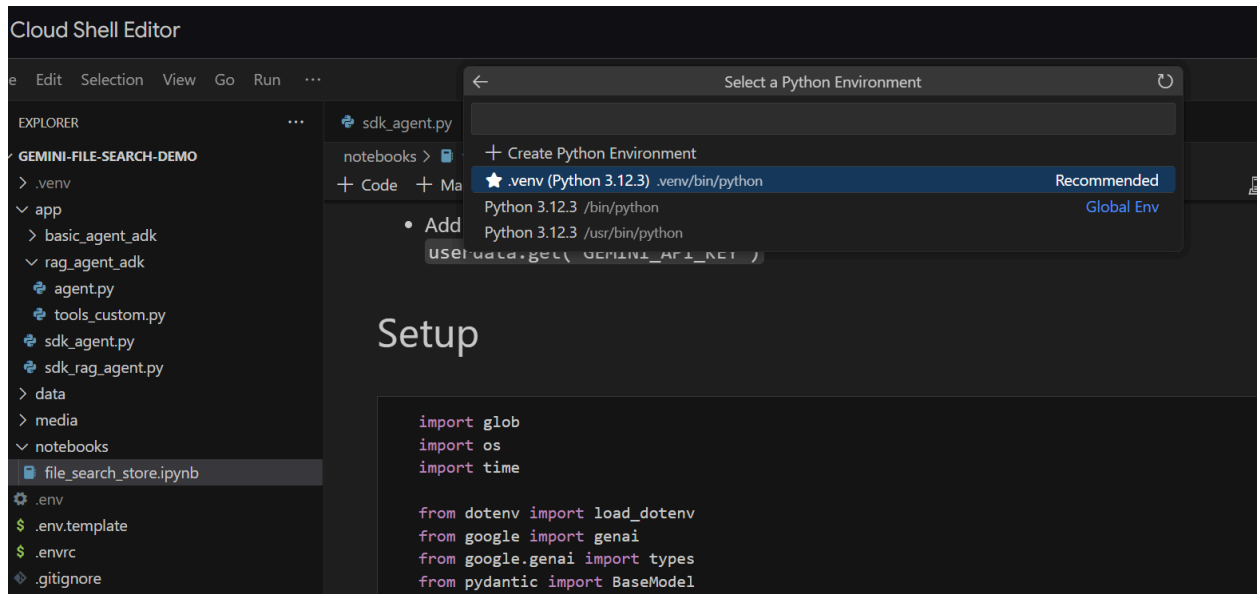
手札情緣

在編輯器中開啟 `notebooks/file_search_store.ipynb` 檔案。如果系統提示您安裝任何 Jupyter VS Code 擴充功能，請繼續安裝。

在 Cloud Shell 編輯器中開啟時，檔案內容如下：



讓我們逐一執行儲存格。請先執行 `Setup` 儲存格，匯入所需項目。如果先前未執行過筆記本，系統會要求您安裝必要擴充功能。請繼續這麼做。接著，系統會要求你選取核心。選取「`Python environments...`」，然後選取我們在稍早執行 `make install` 時安裝的本機 `.env`：



然後：

- 執行「**Local Only**」儲存格，以提取環境變數。
- 執行「Client Initialisation」(用戶端初始化) 儲存格，初始化 Gemini 生成式 AI 用戶端。
- 使用輔助函式依名稱擷取 Gemini 檔案搜尋儲存庫，然後執行「Retrieve the Store」(擷取儲存庫) 儲存格。

現在可以開始建立商店。

- 執行「Create the Store (One Time)」儲存格，建立商店。這項操作只需要執行一次。如果程式碼執行成功，您應該會看到 `"Created store: fileSearchStores/<someid>"` 訊息
- 執行「**View the Store**」(查看商店) 儲存格，查看商店內容。此時，您應該會看到該集合包含 0 份文件。

太好了！現在 Gemini 檔案搜尋儲存庫已準備就緒。

上傳資料

我們想將 `data/story.md` 上傳至商店。請完成下列步驟：

- 執行設定上傳路徑的儲存格。這會指向我們的 `data/` 資料夾。
- 執行下一個儲存格，為將檔案上傳至儲存庫建立公用函式。請注意，這個儲存格中的程式碼也會使用 Gemini 從每個上傳的檔案擷取中繼資料。我們會擷取這些值，並以自訂中繼資料的形式儲存在商店中。(您不必執行這項操作，但這麼做很有幫助)。
- 執行儲存格來上傳檔案。請注意，如果我們先前已上傳同名檔案，筆記本會先刪除現有版本，再上傳新版本。畫面上應會顯示檔案已上傳的訊息。

讓我們詢問前一位服務專員無法回答的問題：

```
> Who pilots the 'Too Many Pies' ship?
```

結果如何？

```
2026-01-10 21:22:25,649 - INFO - Found store: fileSearchStores/demofilestore-5fayifsabstz
Connected to Store: demo-file-store
Type 'exit' or 'quit' to stop.
-----
User: Who pilots the 'Too Many Pies' ship?
2026-01-10 21:22:53,785 - INFO - AFC is enabled with max remote calls: 10.
2026-01-10 21:22:57,366 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:
generateContent "HTTP/1.1 200 OK"
Agent: The 'Too Many Pies' ship is an AI-piloted starship that is part of Commander Dazbo's squadron. It is described as a se
ntient vessel with its own distinct personality. 'Too Many Pies' is an Anaconda-class ship, functioning as a "flying fortress
" and is a significant investment at 365 million credits. It has a jump range of 31.4 LY and is known for carrying munitions.
The AI of the ship also has a fondness for jam tarts.

[Grounding] Found 5 chunks from File Search.
User: 
```

成功！從回覆中，我們可以得知：

- 我們使用檔案儲存庫來回答問題。
- 找到 5 個相關區塊。
- 答案完全正確！

輸入 `quit` 即可關閉代理程式。

7. 將代理轉換為使用 ADK

Google [Agent Development Kit \(ADK\)](#) 是開放原始碼的模組化架構和 SDK，可供開發人員建構代理和代理功能系統。可輕鬆建立及自動調度管理多代理系統。ADK 雖然已針對 Gemini 和 Google 生態系統最佳化，但不受模型和部署方式限制，且可與其他架構相容。如果您尚未用過 ADK，請前往 [ADK 文件](#) 瞭解詳情。

使用 Google 搜尋的 Basic ADK 代理

請看一下 `app/basic_agent_adk/agent.py`。在這個程式碼範例中，您可以看到我們實際實作了兩個代理程式：

1. `root_agent`，可處理與使用者的互動，並提供主要系統指令。
2. 使用 `google.adk.tools.google_search` 做為工具的獨立 `SearchAgent`。

`root_agent` 實際上會使用 `SearchAgent` 做為工具，並透過下列程式碼行實作：

```
tools=[AgentTool(agent=search_agent)],
```

根代理程式的系統提示如下所示：

```
You are a helpful AI assistant designed to provide accurate and useful information.
If you don't know the answer, use the SearchAgent to perform a Google search.
Do not attempt to search more than ONCE.
If the search yields no relevant results or returns unrelated content, you MUST immediately
respond with: "I could not find any information about that."
Do NOT retry the search with different terms. Do NOT ask for clarification. FAIL FAST.
```

試用代理

ADK 提供多種現成介面，方便開發人員測試 ADK 代理。其中一個介面是網頁 UI。這樣我們就能在瀏覽器中測試代理，完全不必編寫任何使用者介面程式碼！

執行下列指令即可啟動這個介面：

```
make adk-playground

# Or, without make:
uv run adk web app --port 8501 --reload_agents
```

請注意，這項指令會將 `adk web` 工具指向 `app` 資料夾，工具會自動探索實作 `root_agent` 的任何 ADK 代理程式。我們來試試看：

```

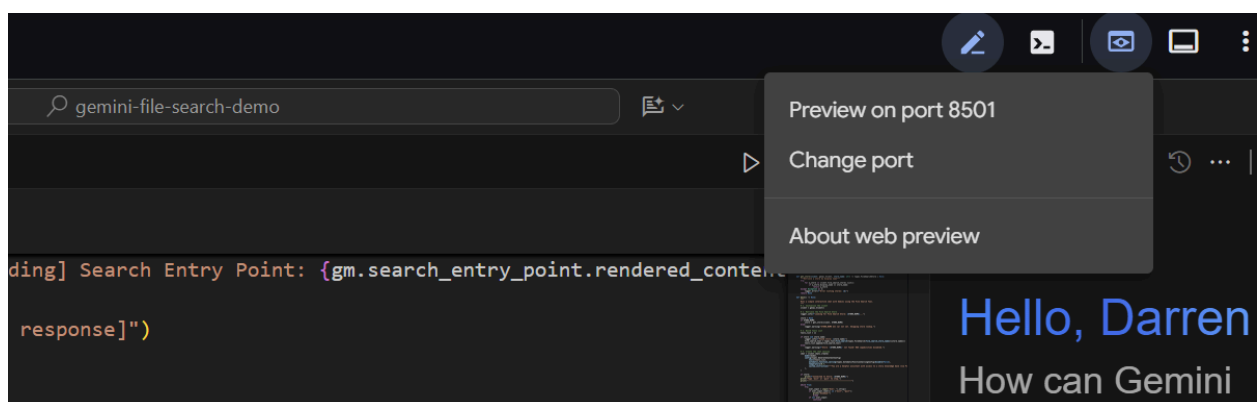
derailed_dash@cloudshell:~/gemini-file-search-demo$ make adk-playground
=====
| 🚀 Starting your agent playground... |
=====
uv run adk web app --port 8501 --reload_agents
/home/derailed_dash/gemini-file-search-demo/.venv/lib/python3.12/site-packages/google/adk/cli/fast_api.py:159: UserWarning: [
EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without
notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/home/derailed_dash/gemini-file-search-demo/.venv/lib/python3.12/site-packages/google/adk/auth/credential_service/in_memory_c
redential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be
removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO:      Started server process [2444]
INFO:      Waiting for application startup.

+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://127.0.0.1:8501. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8501 (Press CTRL+C to quit)

```

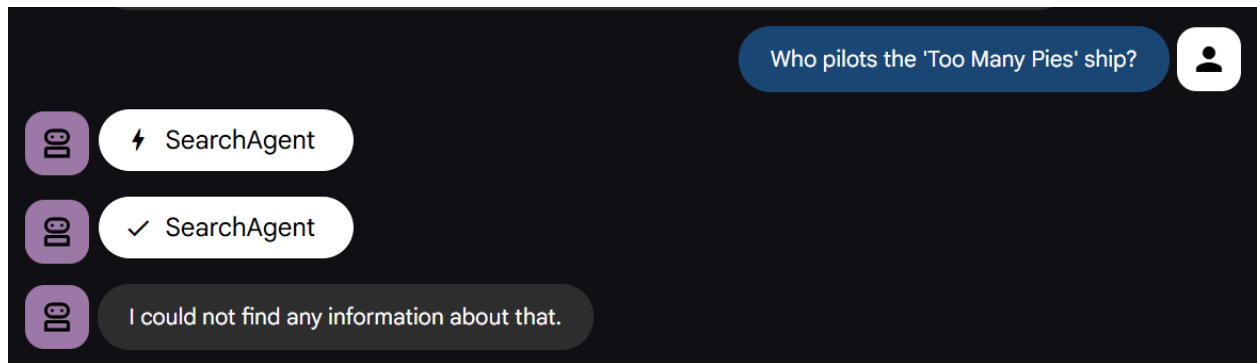
幾秒後，應用程式就會準備就緒。如果您在本機執行程式碼，只要將瀏覽器指向 `http://127.0.0.1:8501` 即可。如果您在 Cloud Shell 編輯器中執行，請按一下「**Web preview**」，然後將通訊埠變更為 **8501**：



使用者介面顯示後，從下拉式選單中選取 `basic_agent_adk`，然後即可向 AI 提問：

到目前為止都很順利！網頁 UI 甚至會顯示根代理程式何時委派給 `SearchAgent`。這項功能非常實用。

現在，請提出需要瞭解故事內容的問題：



歡迎親自試試。您應該會發現，系統會按照指示快速失敗。

將檔案搜尋儲存庫併入 ADK 代理

現在我們要將所有內容整合在一起。我們將執行 ADK 代理程式，該程式可同時使用檔案搜尋儲存庫和 Google 搜尋。查看 `app/rag_agent_adk/agent.py` 中的程式碼。

這段程式碼與上一個範例類似，但有幾項主要差異：

1. 我們有一個根代理，可調度兩個專用代理：
2. **RagAgent**：專屬知識專家 - 使用我們的 Gemini 檔案搜尋儲存庫
3. **SearchAgent**：一般知識專家 - 使用 Google 搜尋
4. 由於 ADK 尚未內建 `FileSearch` 的 wrapper，我們使用自訂 wrapper 類別 `FileSearchTool` 包裝 `FileSearch` 工具，將 `file_search_store_names` 設定插入低階模型要求。這項功能已實作到個別指令碼 `app/rag_agent_adk/tools_custom.py` 中。

此外，還有一個「陷阱」需要注意。撰寫本文時，您無法在同一個要求中，對同一個代理程式使用原生 `GoogleSearch` 工具和 `FileSearch` 工具。如果嘗試這麼做，就會收到類似以下的錯誤訊息：

錯誤 - 發生錯誤：400 INVALID_ARGUMENT。{'error': {'code': 400, 'message': 'Search as a tool and file search tool are not supported together', 'status': 'INVALID_ARGUMENT'}}

錯誤：400 INVALID_ARGUMENT。{'error': {'code': 400, 'message': 'Search as a tool and file search tool are not supported together', 'status': 'INVALID_ARGUMENT'}}

解決方法是將這兩位專家代理程式實作為個別的子代理程式，並使用「代理程式即工具」模式將其傳遞至根代理程式。最重要的是，根層級代理程式的系統指令會提供非常明確的指引，首先使用 `RagAgent`：

你是有用的 AI 助理，可提供準確實用的資訊。

你可諮詢兩位專員：

1. `RagAgent`：從內部知識庫取得專屬資訊。
2. `SearchAgent`：從 Google 搜尋取得一般資訊。

請務必先嘗試 `RagAgent`。如果無法獲得實用答案，請嘗試使用 `SearchAgent`。

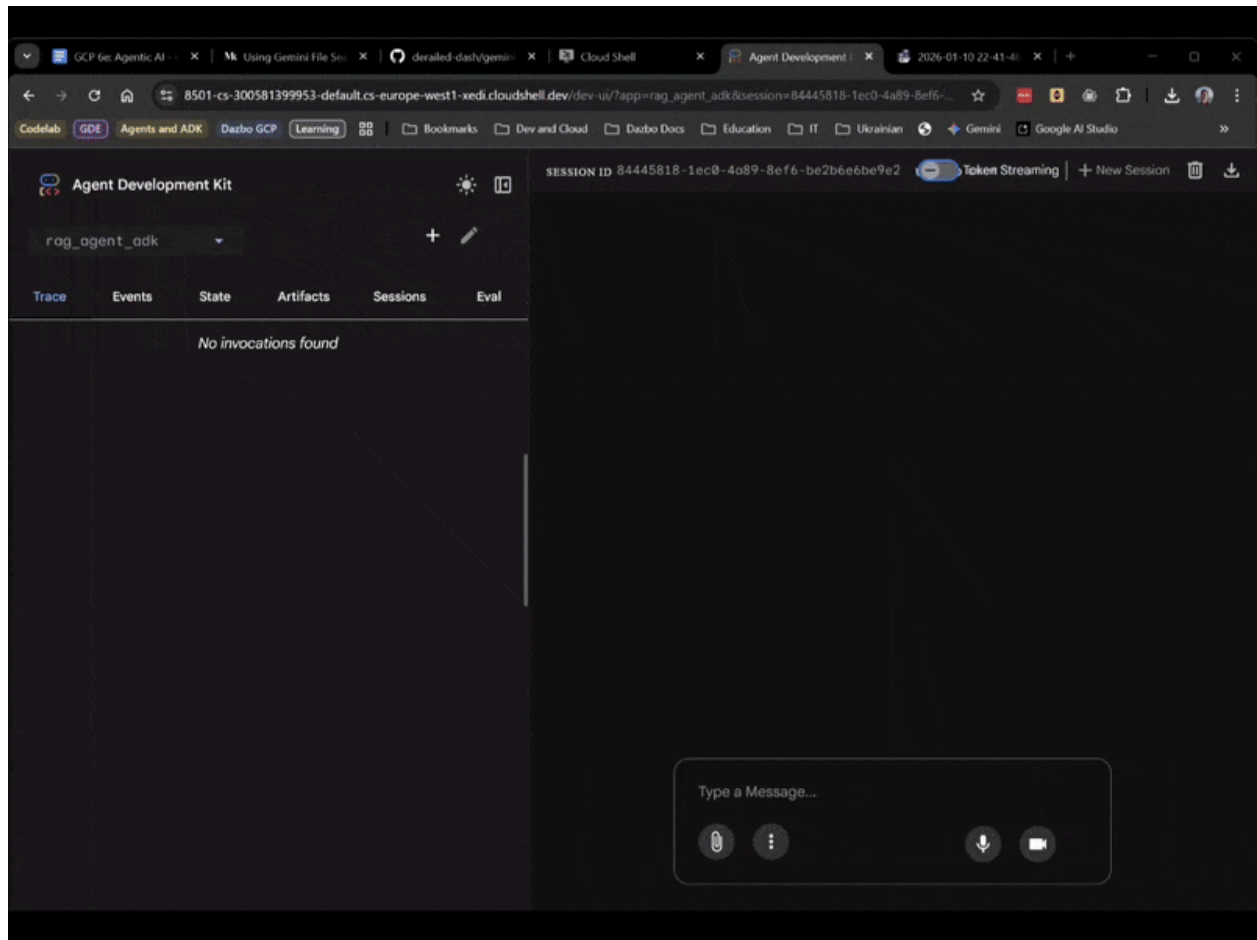
最終測試

如先前所述執行 ADK 網頁 UI：

```
make adk-playground

# Or, without make:
uv run adk web app --port 8501 --reload_agents
```

這次請在 UI 中選取 `rag_agent_adk`。實際運作情況：



從畫面中可以看到，系統會根據問題選擇適當的子代理。

步驟 8 · 約 1 分鐘

8. 結論

恭喜您完成本程式碼研究室！

您已從簡單的指令碼，變成支援 RAG 的多代理系統，完全不需要任何嵌入程式碼，也不必實作向量資料庫！

我們學到：

- **Gemini 檔案搜尋**是代管式 RAG 解決方案，可節省時間和精力。
- **ADK** 提供複雜多代理程式應用程式所需的結構，並透過 Web UI 等介面，為開發人員帶來便利。
- **"Agent-as-a-Tool"**模式可解決工具相容性問題。

希望這個實驗室對您有幫助。下次見！
